

# C<sup>2</sup>KV: Compressed and Composable KV Cache Reuse for Efficient LLM Inference

Chuheng Du  
School of Computer Science  
Shanghai Jiao Tong University  
Shanghai, China  
dch7723@sjtu.edu.cn

Junyi Chen  
School of Computer Science  
Shanghai Jiao Tong University  
Shanghai, China  
junyi.chen@sjtu.edu.cn

Hanlin Tang  
Alibaba Group  
Hangzhou, China  
tanghl1994@gmail.com

Kan Liu  
Alibaba Group  
Hangzhou, China  
liukan.lk@alibaba-inc.com

Tao Lan  
Alibaba Group  
Hang Zhou, Zhejiang Province, China  
tao.lant@alibaba-inc.com

Lin Qu  
Alibaba Group  
Hangzhou, China  
xide.ql@taobao.com

Chaoyue Niu  
School of Computer Science  
Shanghai Jiao Tong University  
Shanghai, China  
rvince@sjtu.edu.cn

Shengzhong Liu\*

School of Computer Science  
Shanghai Jiao Tong University  
Shanghai, China  
shengzhong@sjtu.edu.cn

Guihai Chen  
School of Computer Science  
Shanghai Jiao Tong University  
Shanghai, China  
gchen@cs.sjtu.edu.cn

Fan Wu  
School of Computer Science  
Shanghai Jiao Tong University  
Shanghai, China  
fwu@cs.sjtu.edu.cn

## Abstract

Long-context inference is central to modern large language model (LLM) applications such as retrieval-augmented generation. To mitigate the growing inference cost, recent work has explored non-prefix key-value (KV) cache reuse to reduce redundant prefill computation. However, existing reuse methods primarily focus on computation savings and overlook a critical bottleneck in long-context LLM serving: the cost of storing and accessing large KV caches. While KV compression appears to be a natural complement, naively combining compression with non-prefix KV reuse often leads to severe accuracy degradation. In this work, we propose C<sup>2</sup>KV, a unified framework for non-prefix KV reuse that jointly optimizes KV cache compression and concatenation. C<sup>2</sup>KV learns a composable and compressed KV cache manifold that is explicitly designed to be position-agnostic. Our approach introduces a lightweight sidecar Extractor with learnable compression tokens and a structured attention flow, enabling modular KV representations that can be flexibly reused and concatenated without modifying the frozen base model. We further employ a compression-concatenation co-training strategy to align extraction-time representations with

their downstream reuse behavior. Extensive experiments across multiple long-context benchmarks and model families demonstrate that C<sup>2</sup>KV significantly reduces KV cache storage and transfer costs, achieving up to 17× inference speedup under long contexts, while preserving generation quality.

## CCS Concepts

• Computing methodologies → Natural language processing.

## Keywords

Large Language Model; Fine-tuning; Token-level Data

## ACM Reference Format:

Chuheng Du, Junyi Chen, Hanlin Tang, Kan Liu, Tao Lan, Lin Qu, Chaoyue Niu, Shengzhong Liu, Guihai Chen, and Fan Wu. 2026. C<sup>2</sup>KV: Compressed and Composable KV Cache Reuse for Efficient LLM Inference. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3817715>

## Resource Availability:

The source code of this paper has been made publicly available at <https://github.com/s7a9/C2KV>.

## 1 Introduction

Large language models (LLMs) have been widely adopted across a broad range of applications, including question answering [4, 6, 39], personal assistants [5, 30] and knowledge-intensive reasoning

\*Shengzhong Liu is the corresponding author.



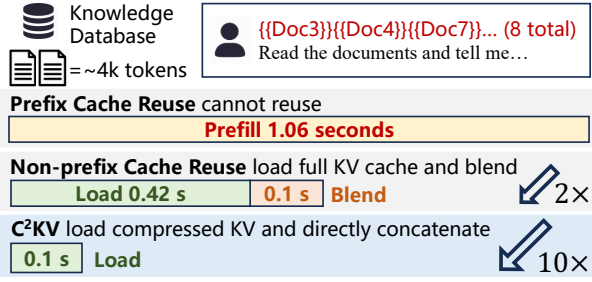
This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

KDD '26, Jeju Island, Republic of Korea

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08

<https://doi.org/10.1145/3770855.3817715>



**Figure 1: Breakdown of Time-to-First-Token (TTFT) in long-context in-context learning (ICL).**

systems [45]. In practical deployments, these applications are increasingly powered by *in-context learning* (ICL) paradigms such as retrieval-augmented generation (RAG) [8, 12, 16, 19], agent systems with long-term memory [29], and few-shot learning [3]. A common characteristic of these workloads is the continuous growth of input context length [7, 22, 31], as each request injects multiple long documents into the model input.

As context lengths increase, LLM inference becomes prohibitively expensive. Given an input sequence of length  $L$ , inference incurs a compute-intensive prefill stage with  $O(n^2)$  complexity to construct key-value (KV) caches, followed by autoregressive decoding where each generation step attends to all cached tokens, resulting in  $O(L)$  memory storage and bandwidth costs. Consequently, long-context inference simultaneously stresses GPU compute, memory capacity, and memory bandwidth, motivating the need for effective KV cache reuse mechanisms in LLM serving systems.

KV cache reuse has therefore attracted significant attention. Existing approaches can be broadly categorized into three classes. 1) *Prefix-based reuse* exploits identical prefixes across requests and enables exact KV reuse under standard causal attention; however, its applicability is severely limited in ICL workloads where repeated documents rarely appear at the same position. 2) To go beyond prefix matching, *training-free non-prefix* methods attempt to concatenate independently computed KV caches with limited recomputation to mitigate context mismatch, reducing prefill computation while preserving the original model. 3) In contrast, *training-based* approaches modify the attention structure or introduce auxiliary tokens during pretraining to enable more flexible reuse or compression, at the cost of altering the base model or introducing information bottlenecks.

While existing KV cache reuse techniques are effective at reducing recomputation, they largely overlook a growing bottleneck in long-context LLM inference: the cost of storing and accessing KV caches. In practical deployments, KV caches are often stored outside high-bandwidth accelerator memory and transferred on demand, making KV cache size and memory bandwidth critical factors as context length increases. As a result, previous works overlook a critical challenge in real-world LLM serving systems: **the dominant bottleneck has shifted to KV cache storage and memory bandwidth.**

This naturally motivates incorporating KV compression [14, 28, 41–43] into non-prefix KV reuse. However, this exposes two fundamental challenges. First, most existing compression schemes produce KV representations that are tightly coupled to their original context and thus non-composable, leading to significant accuracy degradation when reused or concatenated across different contexts. Second, there exists an architectural mismatch: long-context serving requires sequence-level compression, whereas many token-eviction methods rely on layer- or head-dependent selection, preventing consistent alignment once contexts are recomposed. As a result, even under the most reuse-compatible setting, **naive compression remains fundamentally incompatible with non-prefix KV reuse**, motivating a composability-aware design.

To resolve this tension, we propose C²KV, a unified framework for non-prefix KV reuse that jointly optimizes the KV extraction and inference-time concatenation, as shown in Figure 1. Unlike prior methods that directly compress or cache the original KV states, which are inherently context-dependent, **C²KV constructs a Composable and Compressed KV cache manifold**. This manifold is specifically designed to be *position-agnostic*, allowing KV segments of various documents to be flexibly “*plugged and played*” at arbitrary positions within a long-context prompt.

The core of C²KV is its C²Extractor, a lightweight sidecar module that operates independently of the frozen base LLM. Unlike traditional fine-tuning, this “attached” module performs extraction without perturbing the model’s original parametric knowledge. To achieve efficient reuse, the C²Extractor injects learnable C²Tokens as compressed memory slots, whose interactions are governed by a novel *Structured Attention Flow*. This mechanism serves a dual purpose: first, it acts as a high-fidelity semantic aggregator that maximizes information retention via localized grouping; second, it ensures the resulting KV representations are modular and position-independent. This structured flow inherently facilitates seamless downstream concatenation while eliminating semantic cross-talk.

To further fortify the robustness of these compressed representations, we employ a *Compression-Concatenation Co-training* strategy. By supervising the model on generation tasks conditioned on concatenated KV segments, we implicitly steer the C²Extractor to produce “merge-ready” states that maintain semantic coherence even when reordered. Consequently, C²KV enables aggressive compression with minimal fidelity loss, significantly alleviating the storage and bandwidth bottlenecks of long-context inference.

Our main contributions are summarized as follows:

- We show that the key challenge of KV cache reuse lies in storage and memory transfer overheads, whereas existing approaches only reduce recomputation and ignore the storage overhead.
- We propose a unified training formulation that jointly optimizes KV extraction and inference-time KV concatenation, explicitly aligning the extracted representations with their reuse behavior under non-prefix inference.
- We introduce a new attention information flow through a carefully designed attention mask to preserve generation accuracy when concatenating compressed KV caches.
- Extensive experiments show that our method significantly reduces KV cache storage and transfer cost, achieving up to 17× inference speedups over existing approaches, without degrading the inference quality.

## 2 Preliminaries and Motivation

### 2.1 KV Cache in LLM Inference

The Transformer-based Large Language Model (LLM) generates tokens auto-regressively. To avoid redundant computations, Key-Value (KV) caching stores the intermediate attention states. Formally, for a sequence  $\{x_1, \dots, x_t\}$ , the KV cache at layer  $l$  is:

$$\mathbf{K}_l = [k_1, \dots, k_t], \quad \mathbf{V}_l = [v_1, \dots, v_t]. \quad (1)$$

During the generation of  $x_{t+1}$ , the model only computes the KV pair for the new token and concatenates it with the existing cache. While accelerating standard inference, the sequential dependency of self-attention poses a challenge for reusing KV blocks in different contexts.

### 2.2 Generalized KV Cache Reuse

Existing LLM serving systems primarily utilize Prefix Caching (e.g., vLLM [18], RadixAttention [44]), which allows the reuse of KV states if the leading sequence of a new request exactly matches a previously cached one. This is achieved by simple position-aligned mapping. However, in complex scenarios such as multi-document RAG [17] or modular tool-use [24], reusable content (e.g.,  $\text{Doc}_A, \text{Doc}_B$ ) may appear in arbitrary orders or be interleaved with dynamic instructions. To address this, we move beyond prefix matching and consider a Non-Prefix Caching paradigm.

We define a generalized two-stage framework for non-prefix reuse. Given a request with a static context  $\mathbf{x}^s$ , several independent cached documents  $\{\mathbf{x}_1^d, \dots, \mathbf{x}_K^d\}$ , and a query  $\mathbf{x}^q$ , the process is formalized as:

- **Encoding:** An encoder  $\mathcal{M}$  (typically the base LLM itself or a specialized module) extracts KV representations from documents:

$$\mathbf{KV}_k^{\text{reuse}} = \mathcal{M}(\mathbf{x}_k^d). \quad (2)$$

- **Blending:** A blender  $\mathcal{B}$  adapts and integrates these pre-computed states into the new request context:

$$\mathbf{KV}^{\text{total}} = \mathcal{B}(\mathbf{x}^s, \mathbf{KV}_1^{\text{reuse}}, \dots, \mathbf{KV}_K^{\text{reuse}}). \quad (3)$$

- **Inference:** The LLM computes the final output:

$$y = \mathcal{F}([\mathbf{KV}^{\text{total}}, \mathbf{x}^q]). \quad (4)$$

Based on the implementation of  $\mathcal{M}$  and  $\mathcal{B}$ , current methods fall into two categories:

- **Training-Free Methods:** These methods (e.g., CacheBlend [38], KVShare [35]) use the vanilla LLM as the encoder and employ selective recomputation as the blender to reconcile mismatched KV states.
- **Training-Based Methods:** These methods (e.g., Block-Attention [26], KVLink [36]) modify the model’s architecture or fine-tune its weights so that it natively learns to process disjoint or reordered KV blocks.

### 2.3 Limitations of Existing Reuse Methods

Despite their potential, both categories of methods face critical bottlenecks that prevent efficient and lossless serving.

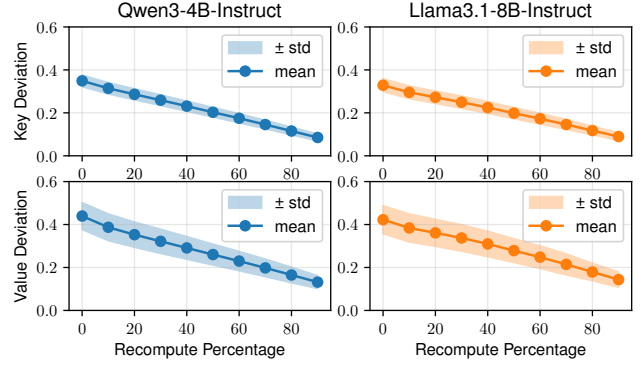


Figure 2: KV Deviation grows as recompute ratio drops.

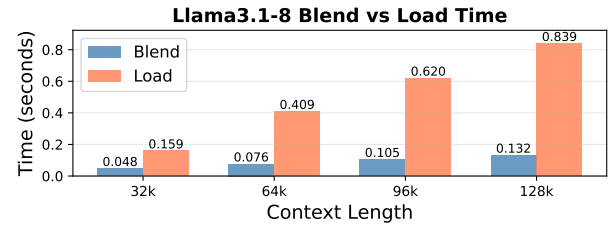


Figure 3: Latency comparison of blending and loading.

**2.3.1 The KV Deviation Gap in Training-Free Methods.** In training-free methods, the blender  $\mathcal{B}$  identifies a subset of tokens  $M(\mathbf{x})$  to recompute. However, since the remaining unrecomputed KV states were generated in a different context, there exists an inherent KV deviation ( $\Delta\text{KV}$ ):

$$\Delta\text{KV} = \|\mathbf{KV}^{\text{full}} - \mathbf{KV}^{\text{blend}}\| / \|\mathbf{KV}^{\text{full}}\| \geq \delta > 0. \quad (5)$$

As shown in Figure 2, this deviation monotonically increases as the recomputation ratio decreases, creating a rigid trade-off between serving latency and model accuracy.

**2.3.2 Generalization Collapse in Training-Based Methods.** Training-based methods attempt to eliminate this gap by altering the self-attention mechanism, which suffers from:

- **Capability Loss:** Altering attention patterns often leads to “catastrophic forgetting”. Our empirical study shows a Llama3.1-8B model fine-tuned with Block-Attention suffers a performance drop of up to 10.4% on LongBench.
- **Inflexibility:** The high cost of per-model fine-tuning makes it difficult to adapt these methods to the rapid iteration of open-source base models.

**2.3.3 Storage and Bandwidth Bottlenecks.** Crucially, prior methods primarily focus on reducing blending computation, while overlooking the fact that modern long-context serving is increasingly **memory-bound**. As context length  $L$  grows, the KV cache size scales linearly, making the transfer of uncompressed KV states between memory hierarchies (e.g., from DRAM to HBM) the dominant latency factor. As illustrated in Figure 3, existing reuse methods reduce prefill cost by loading cached KVs from memory, but still do not address this storage pressure; moreover, naively applying KV

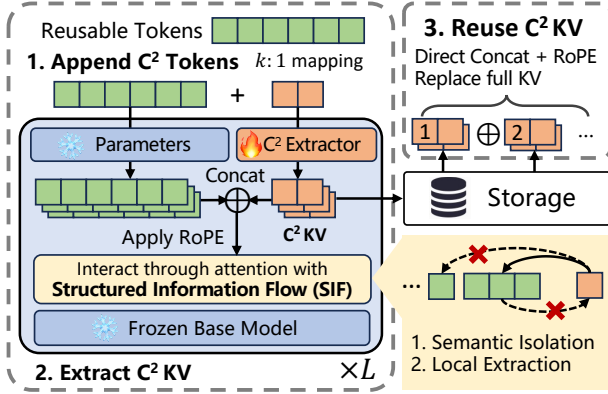


Figure 4: Pipeline overview of C²KV.

compression to reused blocks leads to severe fidelity loss, as standard compressed representations are not designed to be composable or position-agnostic.

### 3 C²KV Framework

#### 3.1 Framework Overview

We propose a non-prefix KV cache reuse framework C²KV that enables long documents to be compressed, stored, and directly reused at arbitrary positions. C²KV augments a frozen base LLM with a lightweight trainable module, together forming the C²Extractor, which produces compact and composable KV representations.

The pipeline consists of two stages. First, given a set of documents, the C²Extractor encodes each document independently into a compressed, position-agnostic KV cache, referred to as C²KV. The C²Extractor employs Structured Information Flow to preserve the base model’s semantics while enabling compression. Second, at inference time, the requested C²KV caches are retrieved, assigned positional embeddings according to their designated position, and directly concatenated to condition generation.

To ensure the validity of the extracted KV caches after concatenation, the C²Extractor is trained using Compression-Concatenation Co-Training, jointly optimizing compression and generation under a single supervised objective. This design decouples document encoding from query-time generation, enabling efficient non-prefix reuse with reduced KV cache size and minimal impact on generation quality. An overview of the pipeline is illustrated in Figure 4.

#### 3.2 C²Extractor

The C²Extractor is a lightweight module attached to a frozen base language model. It takes a document as input and produces a compressed, composable key-value representation, whose length is reduced by a  $k : 1$  ratio. Given a document token sequence, the C²Extractor outputs per-layer key-value pairs, referred to as C²KVs, which are stored and later retrieved for downstream generation.

**3.2.1 C²Tokens as Memory Slots.** Compression is achieved by introducing a sequence of auxiliary tokens, referred to as C²Tokens, into the document token stream. Given a compression ratio  $k$ , the number of C²Tokens is determined as  $m = \lceil n/k \rceil$ . The C²Tokens are logically interleaved with the document tokens such that one C²Token is associated with each consecutive block of  $k$  document

tokens. All C²Tokens share the same learnable embedding vector and do not correspond to any lexical content.

**3.2.2 New QKV Projection Heads.** The core design of the C²Extractor is a set of new, per-layer QKV projection heads that operate exclusively on the C²Tokens. For each Transformer layer  $\ell$ , the base model’s original QKV projections are kept frozen and are applied only to the original document tokens. In contrast, C²Tokens are projected using separate trainable matrices:

$$Q_{C^2}^{(\ell)} = W_{C^2, Q}^{(\ell)} h_{C^2}^{(\ell)}, \quad K_{C^2}^{(\ell)} = W_{C^2, K}^{(\ell)} h_{C^2}^{(\ell)}, \quad V_{C^2}^{(\ell)} = W_{C^2, V}^{(\ell)} h_{C^2}^{(\ell)}, \quad (6)$$

where the projection dimensions exactly match those of the base model, ensuring compatibility with downstream attention operations.

The projected C²QKV tensors are concatenated with the frozen QKV tensors of the original document tokens and passed into the standard attention operator. Only the parameters of the C²Token embeddings and their associated QKV projection heads are trainable; all base model parameters remain frozen. This design adds around 10% of additional parameters to the base model for Qwen3.4B and Llama3.1-8B.

**3.2.3 Residual Mean-Pooling Variant.** We also consider a variant of the Extractor that incorporates residual semantic aggregation, denoted as C²KV-Residual. In this variant, at the first Transformer layer, we compute the mean of the hidden states of the  $k$  document tokens associated with a given C²Token and add it to the C²Token’s hidden state:

$$h_{C^2}^{(\ell)} \leftarrow h_{C^2}^{(\ell)} + \frac{1}{k} \sum_{i \in \mathcal{B}} h_i^{(\ell)}, \quad (7)$$

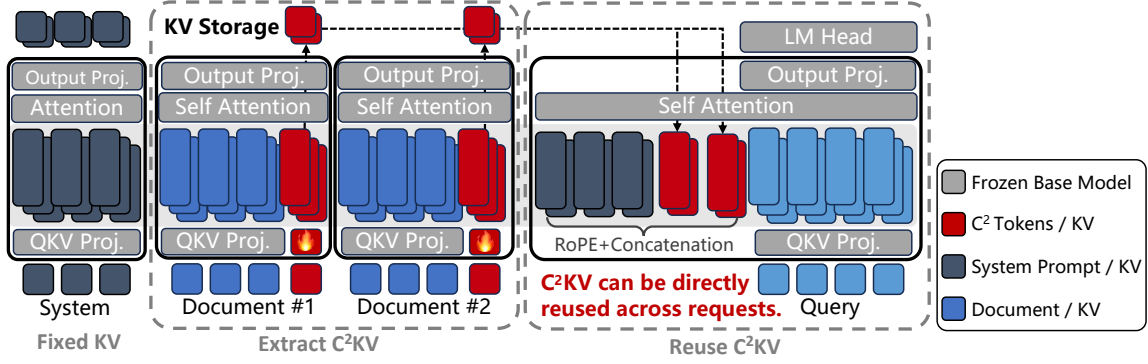
where  $\mathcal{B}$  denotes the corresponding document token block. The scaling factor is fixed to 1.0. This residual pathway provides an explicit aggregation of local document semantics and improves robustness under compression.

**3.2.4 Output and Reuse.** After extraction, only the KV pairs associated with the C²Tokens are retained as C²Extractor KVs. During inference, the retrieved KVs are applied new positional embeddings according to their placement in the final input sequence, where positional offsets are accumulated across concatenated segments and each compressed KV segment is treated as occupying the same effective span as its original uncompressed tokens. The compressed KV segments are then directly concatenated and used to condition downstream generation, without reintroducing the original tokens.

#### 3.3 Structured Information Flow

The goal of Structured Information Flow (SIF) is to enable **semantic extraction and compression without altering the behavior of the frozen base model**. To this end, we impose a block-structured attention pattern that strictly separates the information pathways of Original Tokens and C²Tokens during extraction.

**3.3.1 Token layout and block structure.** Given a document token sequence  $x_{1:n}$ , we partition it into contiguous blocks of size  $k$ . For each block  $\mathcal{B}(j) = \{(j-1)k+1, \dots, \min(jk, n)\}$ , we introduce exactly one C²Token  $c_j$ . Conceptually,  $c_j$  is placed after all tokens in its corresponding block, so that standard causal attention suffices to enforce block-local visibility. For implementation convenience,

Figure 5: Architecture overview of C<sup>2</sup>KV.

all C<sup>2</sup> Tokens are physically grouped after the document tokens, and logical block boundaries are enforced solely through the attention mask. Each C<sup>2</sup> Token is assigned the same position index as the last token in its block. This positional assignment is used only during extraction; the resulting key-value pairs are stored before positional embeddings are applied.

**3.3.2 Attention constraints.** At every layer, we apply an attention mask that enforces the following constraints.

- **Original-token invariance.** Original Tokens are allowed to attend only to **other Original Tokens** under the standard causal mask. Attention from Original Tokens to any C<sup>2</sup> Token is strictly disallowed:

$$M_{o,c} = -\infty \quad \forall o \in \mathcal{O}, c \in \mathcal{C}. \quad (8)$$

As a result, the hidden states and KV caches of Original Tokens are identical to those produced by the frozen base model.

- **Block-local semantic extraction.** Each C<sup>2</sup> Token  $c_j$  may attend only to the Original Tokens in its own block  $\mathcal{B}(j)$  and the sink block  $\mathcal{B}(0)$ :

$$M_{c_j,o} = 0 \quad \forall o \in \mathcal{B}(j) \cup \mathcal{B}(0), M_{c_j,o} = -\infty \quad \forall o \notin \mathcal{B}(j) \cup \mathcal{B}(0). \quad (9)$$

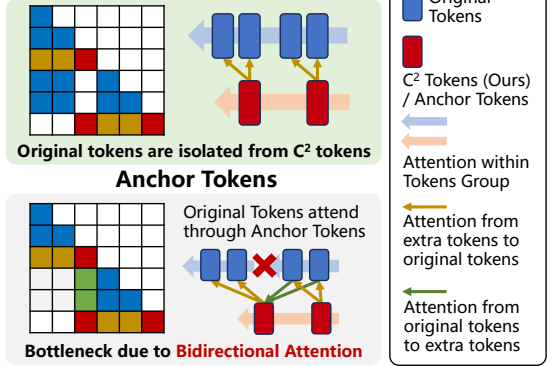
Since  $c_j$  is conceptually positioned at the end of the block, causal masking guarantees that it can attend to all tokens within  $\mathcal{B}(j)$  while preventing access to tokens beyond the block boundary. This enforces a  $k:1$  structure and prevents global information leakage during extraction.

- **Causal accumulation across blocks.** To allow compressed representations to accumulate document-level context, C<sup>2</sup> Tokens may attend to preceding C<sup>2</sup> Tokens causally. This enables later blocks to incorporate information from earlier compressed segments, while preserving autoregressive consistency.

### 3.3.3 Comparison with Anchor Token-based Compression.

Figure 6 contrasts our Structured Attention Flow with anchor token-based compression methods, which also interleave auxiliary tokens into the input sequence. While both approaches introduce extra tokens, they serve fundamentally different purposes. Anchor tokens are designed as semantic bottlenecks for context compression and rely on bidirectional attention between original tokens and anchor tokens. As a result, original token representations are modified through anchor tokens, and the resulting KV cache is inherently entangled with the full input context.

### Structured Attention Flow (Ours)

Figure 6: Comparison between Structured Attention Flow with C<sup>2</sup> Tokens and anchor token-based compression. The sink block in block-local extraction is omitted for simplicity.

In contrast, C<sup>2</sup> Tokens are introduced as latent KV carriers rather than semantic bottlenecks. Information flows only from original tokens to C<sup>2</sup> Tokens in a block-local and causal manner. This asymmetry is critical: it allows the extracted C<sup>2</sup> KVs to be context-independent, composable, and directly reusable across requests. Consequently, although anchor token-based methods are effective for compressing context within a single input, they are fundamentally incompatible with non-prefix KV cache reuse. Our design replaces semantic compression with structural KV extraction, enabling reuse by construction rather than approximation.

## 3.4 Composable KV Learning via Compression-Concatenation Training

The objective of compression-concatenation training is to ensure that document-level compression and KV cache concatenation are , so that independently extracted KV caches remain valid and useful after being composed in arbitrary multi-document contexts.

**3.4.1 Training objective.** The entire model is trained using a single supervised fine-tuning objective on the generated answer. Let  $\mathbf{K}_{\text{cat}}, \mathbf{V}_{\text{cat}}$  denote the concatenated document KV cache, and let  $q_{1:m}$  and  $y_{1:T}$  denote the query and answer tokens. The training loss is the standard autoregressive language modeling loss:

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^T \log p(y_t | y_{<t}, q_{1:m}, \mathbf{K}_{\text{cat}}, \mathbf{V}_{\text{cat}}). \quad (10)$$

No auxiliary reconstruction loss, alignment loss, or document-level supervision is introduced. Gradients are propagated exclusively to the parameters of the  $C^2$ Extractor, including the  $C^2$ Token embeddings and their dedicated QKV projection heads; all base model parameters remain frozen.

**3.4.2 Why concatenation is learned.** Although documents are extracted independently, supervision is applied only after concatenation. Each document’s  $C^2$ KV cache is optimized based on its contribution to answering the query in the presence of other documents. Since document order and the number of documents vary across training samples, the Extractor is forced to project document semantics into a KV cache manifold that remains valid under different compositions. Importantly, compression and concatenation are not optimized in isolation. A document KV cache that compresses too aggressively but fails after concatenation will incur higher generation loss, as will a KV cache that only functions in a fixed global context. Through end-to-end supervision, the Extractor learns a representation that balances both requirements and maps document tokens into a compression concatenation enabled KV manifold.

## 4 Evaluations

### 4.1 Experiment Setup

**4.1.1 Dataset.** We train the  $C^2$ Extractor using multi-document supervised fine-tuning (SFT) question-answering data, while keeping all Base Model parameters frozen. The training corpus is constructed from the training splits of three publicly available QA datasets: HotpotQA [37], 2WikiMultiHopQA [13], and LongMagpie [9]. Together, these datasets cover a wide range of multi-document reasoning scenarios, making them suitable for learning document-level KV extraction under non-prefix reuse settings.

Each instance is formatted using the target model’s official chat template, including a system prompt, documents, user question, and reference answer. To improve formatting robustness and reduce verbosity, both system prompts and QA instructions are randomly sampled from a small prompt pool. During extraction, document tokens do not attend to system tokens; system KV’s are concatenated only at generation time.

For evaluation, we use the LongBench [40] benchmarks. For the single-document subsets in LongBench, we partition each document into natural paragraphs and treat them as multiple documents, enabling evaluation of non-prefix KV reuse.

**4.1.2 Metrics.** We adopt task-appropriate evaluation metrics following prior works. For question-answering tasks, we report the **F1 score**, which measures token-level overlap between the predicted answers and the ground-truth references. For summarization tasks, we report **ROUGE-L**, which evaluates sequence-level similarity based on the longest common subsequence between the generated and reference summaries.

**4.1.3 Models.** We conduct experiments using three widely adopted instruction-tuned large language models: Qwen3-4B-Instruct-2507 [33], Llama-3.1-8B-Instruct [11], and Qwen2.5-7B-Instruct [34]. These models cover different architectures and parameter scales, allowing us to evaluate the robustness of our method across model families.

**4.1.4 Baselines.** We compare  $C^2$ KV against 5 baselines:

- **Full KV Recompute (FR):** The model performs a full prefill for every request without any KV reuse, serving as an accuracy and computation upper bound;
- **Full KV Reuse (Reuse):** KV caches of reusable documents are directly reused without correction or retraining, representing an aggressive but error-prone lower bound;
- **EPIC** [15]: A selective recompute reuse method that choose tokens by a static pattern;
- **CacheBlend** [38]: A selective recompute reuse method that recomputes tokens by value differences to mitigate errors.
- **Block-Attention** [26]: A training-based baseline by continuing training with no cross-attention between documents.

### 4.2 TTFT-Accuracy Trade-off

**4.2.1 Experiment setup.** In this experiment, we focus on Time-to-First-Token (TTFT) under a document-reuse serving scenario, where a document corpus is available and document KV caches can be pre-extracted offline. TTFT is defined as the time elapsed from receiving a user request to the moment when the language model has its past KV cache fully prepared on GPU and is ready to start decoding the first output token.

Under this definition, different methods incur TTFT from different sources:

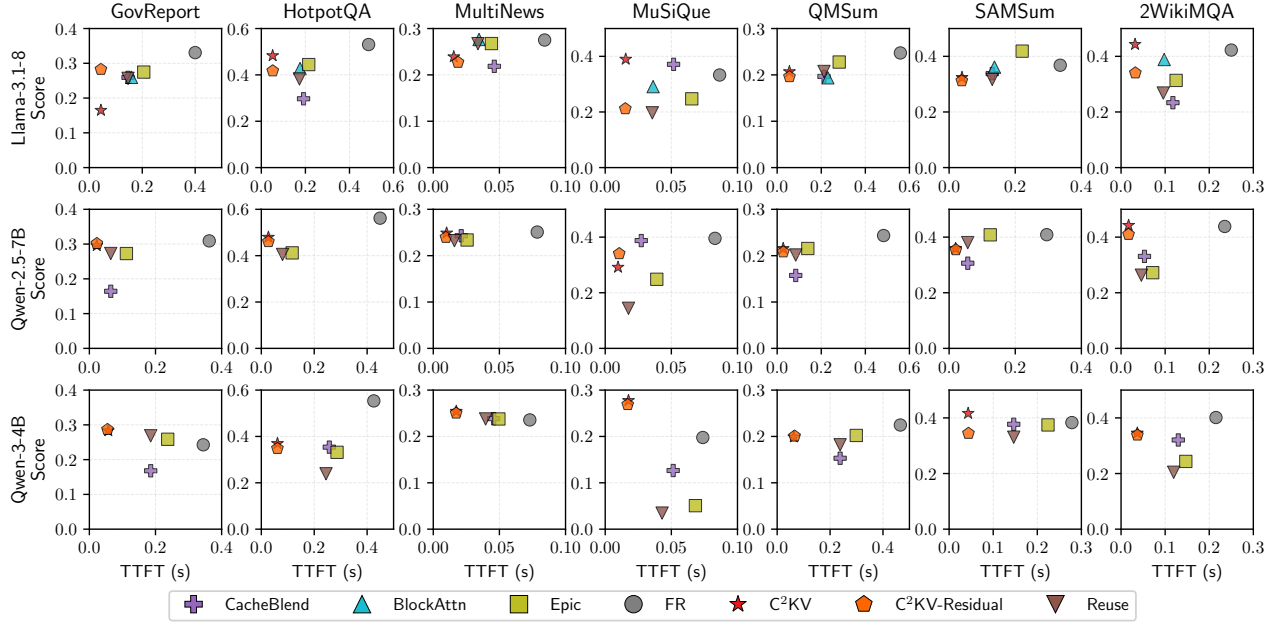
- Full Recompute (FR) must prefill the entire document context online and thus its TTFT corresponds to the full-context prefill latency.
- Non-prefix reuse methods load precomputed document KV’s from host memory and additionally perform blending or structural transformations before decoding.
- $C^2$ KV loads compressed document KV’s from host memory and directly reassigns positions without any blending, reducing TTFT to a load-only operation.

Although different reuse methods may employ different KV compression ratios, this difference reflects their inherent design goals. In particular,  $C^2$ KV jointly performs KV compression (4×) and reuse.

**4.2.2 TTFT-Accuracy Trade-off.** Figure 7 shows the trade-off between time-to-first-token (TTFT) and task accuracy across seven LongBench tasks and three instruction-tuned LLMs. Each point corresponds to the average TTFT and task score of a method on the full test set, where lower TTFT and higher accuracy are preferable. This analysis reveals how different KV reuse strategies balance startup latency against generation quality:

**Full recompute offers strong accuracy but poor startup latency.** While FR consistently achieves the highest accuracy by prefilling the entire context online, it suffers from prohibitive TTFT. This establishes a performance ceiling for accuracy but highlights the critical need for efficient KV reuse in long-context workloads.

**Existing non-prefix reuse reduces TTFT but introduces accuracy instability.** Prior non-prefix reuse approaches reduce TTFT (shifting leftward in the plot) but introduce substantial accuracy instability. Methods like EPIC and CacheBlend, which rely on KV blending, require complex preparation and exhibit sensitivity to architectural changes, leading to degraded performance in multi-document reasoning.



**Figure 7: Trade-off between Time-to-First-Token (TTFT, x-axis) and task accuracy (Score, y-axis) under non-prefix document reuse.**

**C<sup>2</sup>KV achieves a superior TTFT-accuracy trade-off.** C<sup>2</sup>KV achieves minimal TTFT and competitive accuracy simultaneously, occupying the optimal upper-left region of the trade-off plot. Its learnable, composable KV representation transforms preparation into a load-only operation, eliminating errors from blending. While the C<sup>2</sup>KV-Residual variant improves stability on specific tasks, the vanilla C<sup>2</sup>KV delivers the best overall efficiency-performance balance.

Finally, we note that this experiment isolates TTFT, which dominates user-perceived latency for long-context requests. Together with the decode-time results presented in the next section, these findings show that C<sup>2</sup>KV improves both request startup latency and steady-state generation efficiency.

### 4.3 Decode-Time Efficiency

We evaluate decode-stage efficiency using average time per token (TBT), isolating memory bandwidth costs by excluding prefill. All methods use identical decoding settings with KV reuse applied before decoding, and results are shown in Figure 8. Our findings are summarized as follows.

**C<sup>2</sup>KV significantly flattens decode-time scaling with context length.** As context length grows from 16k to 128k tokens, per-token decode latency increases for all methods due to longer attention contexts. Full-length KV caches exhibit near-linear scaling, indicating severe memory bandwidth pressure. In contrast, C<sup>2</sup>KV compresses the KV footprint and largely flattens the decode-time curve, showing only a mild increase even at 128k tokens across all models. Shrinking the effective KV cache size directly translates into lower time-between-tokens (TBT) under long-context decoding.

**Table 1: Performance comparison on Llama3.1-8B across different KV cache methods and compression ratios.** C<sup>2</sup>KV-#x denotes models trained and evaluated with a fixed #x KV compression ratio. C<sup>2</sup>KV-Dyn-Test#x denotes a single model trained with dynamically sampled compression ratios and evaluated under #x inference-time compression ratio.

| LLama3.1-8B                   | MuSiQue | WikiMQA | SAMSum | MultiNews |
|-------------------------------|---------|---------|--------|-----------|
| Full                          | 0.3198  | 0.4018  | 0.3652 | 0.2685    |
| Naïve reuse                   | 0.1970  | 0.2681  | 0.3172 | 0.2674    |
| EPIC                          | 0.2498  | 0.2821  | 0.4217 | 0.2630    |
| CacheBlend                    | 0.2720  | 0.1334  | 0.3379 | 0.2126    |
| C <sup>2</sup> KV-4x          | 0.3587  | 0.4477  | 0.3904 | 0.2532    |
| C <sup>2</sup> KV-8x          | 0.3225  | 0.4462  | 0.2572 | 0.2351    |
| C <sup>2</sup> KV-16x         | 0.2746  | 0.3973  | 0.3556 | 0.2234    |
| C <sup>2</sup> KV-Dyn-4xTest  | 0.3457  | 0.4680  | 0.3779 | 0.2503    |
| C <sup>2</sup> KV-Dyn-8xTest  | 0.3285  | 0.4555  | 0.3694 | 0.2428    |
| C <sup>2</sup> KV-Dyn-10xTest | 0.3198  | 0.4310  | 0.3570 | 0.2357    |
| C <sup>2</sup> KV-Dyn-16xTest | 0.2831  | 0.4298  | 0.3525 | 0.2343    |

**KV compression complements prefill-stage reuse.** Beyond eliminating redundant prefill computation, C<sup>2</sup>KV delivers sustained performance benefits during decoding, where KV access dominates user-perceived latency. These results highlight that non-prefix KV reuse, when combined with effective KV compression, yields system-level gains throughout the entire inference pipeline.

### 4.4 Compression Ratio Scaling

We further evaluate how C<sup>2</sup>KV scales under different KV compression ratios on Llama3.1-8B. Table 1 compares C<sup>2</sup>KV with existing non-prefix reuse methods under both fixed and dynamic compression settings.

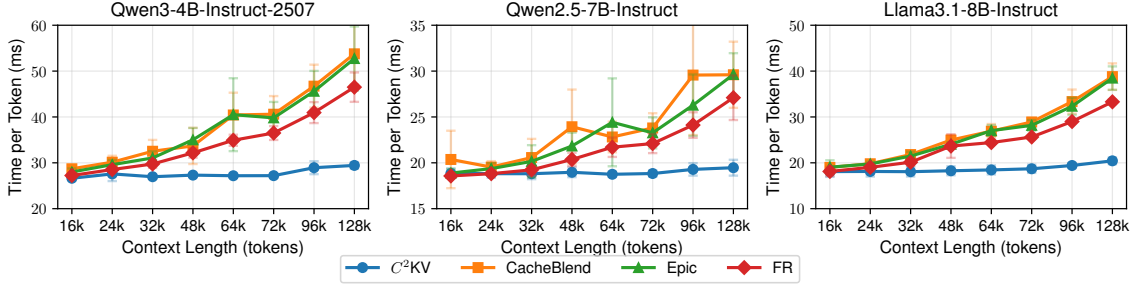


Figure 8: Impact of context length on decode latency across different methods.

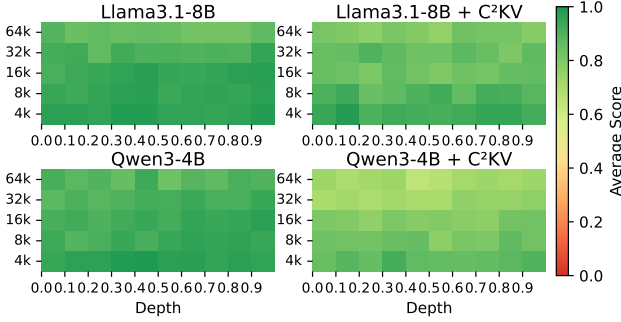


Figure 9: RULER evaluation across different context lengths and answer depths.  $C^2KV$  is tested with  $4\times$  compression ratio. Depth denotes the relative position of the target answer span within the context, where larger depth values correspond to answers appearing later in the sequence.

**Scaling under Fixed Compression Ratios.** As the compression ratio increases from  $4\times$  to  $16\times$ ,  $C^2KV$  exhibits graceful degradation and remains substantially more robust than existing reuse methods. Even under aggressive compression,  $C^2KV$  preserves competitive performance across both QA and summarization tasks, demonstrating that the extracted KV representations remain stable and composable under large sequence reduction ratios.

**Generalization through Dynamic-Ratio Training.** We further train a single model by randomly sampling compression ratios from  $\{4\times, 8\times, 16\times\}$  at each training step. The resulting model generalizes effectively across different inference-time compression budgets, including the unseen  $10\times$  setting. Compared with fixed-ratio training, dynamic-ratio training consistently improves robustness and achieves stronger overall performance across varying compression levels.

Overall, these results demonstrate that  $C^2KV$  provides a unified framework for composable KV reuse and scalable KV compression, while maintaining strong accuracy across a wide range of compression budgets.

#### 4.5 Long-Context Evaluation on RULER

We further evaluate  $C^2KV$  on the RULER benchmark under  $4\times$  KV compression. To support long-context evaluation, each input context is split into segments of approximately 4k tokens before extraction and KV reuse. Figure 9 reports the average retrieval score across different context lengths and answer depths.

Table 2: Performance comparison on Qwen3-14B across different non-prefix KV reuse methods and compression ratios.

| Qwen3-14B               | MuSiQue | WikiMQA | SAMSum | MultiNews |
|-------------------------|---------|---------|--------|-----------|
| Full                    | 0.4384  | 0.5514  | 0.4060 | 0.2472    |
| Naïve reuse             | 0.1046  | 0.3184  | 0.2055 | 0.2421    |
| EPIC                    | 0.2645  | 0.3826  | 0.4093 | 0.2472    |
| CacheBlend              | 0.4187  | 0.3955  | 0.3782 | 0.2413    |
| $C^2KV\text{-}4\times$  | 0.4311  | 0.5625  | 0.4105 | 0.2478    |
| $C^2KV\text{-}8\times$  | 0.4043  | 0.5654  | 0.3835 | 0.2448    |
| $C^2KV\text{-}16\times$ | 0.3342  | 0.5679  | 0.3871 | 0.2403    |

**Stable Long-Context Scaling.**  $C^2KV$  maintains stable retrieval performance as the context length increases from 4k to 64k tokens, demonstrating effective long-range information preservation under compressed KV reuse.

**Robustness under Compression.** Despite operating with  $4\times$  compressed KV caches,  $C^2KV$  achieves only limited degradation compared with the original models, showing that composable KV compression can preserve strong long-context reasoning capability.

#### 4.6 Scaling to Larger Models

To evaluate whether  $C^2KV$  scales to larger model sizes, we further conduct experiments on Qwen3-14B and report the results in Table 2. We compare  $C^2KV$  against existing non-prefix reuse baselines under multiple compression ratios.

The results show that  $C^2KV$  consistently maintains strong performance across all tasks while enabling substantial KV compression. In particular,  $C^2KV\text{-}4\times\text{-Res}$  achieves accuracy close to or exceeding the full-context baseline on MuSiQue, SAMSum, and MultiNews, while significantly outperforming existing reuse methods on multi-document QA tasks such as WikiMQA. Moreover, the performance degradation of  $C^2KV$  remains gradual as the compression ratio increases from  $4\times$  to  $16\times$ , demonstrating favorable scaling behavior under aggressive compression settings.

These results indicate that the composable KV representations learned by  $C^2KV$  generalize effectively to larger LLMs and remain stable under high compression ratios.

#### 4.7 Does Compression Alone Suffice?

To validate the claim that naively combining KV reuse with generic compression degrades accuracy, we conduct a motivating experiment. Table 3 reports LongBench accuracy when a generic KV

**Table 3: Effect of naively combining KV cache compression (SnapKV) with reuse methods on Llama3.1-8B-Instruct.**

| Dataset   | FR     | +SnapKV | Naive  | +SnapKV | BlockAttn | +SnapKV | Epic   | +SnapKV | CacheBlend | +SnapKV |
|-----------|--------|---------|--------|---------|-----------|---------|--------|---------|------------|---------|
| HotpotQA  | 0.5343 | 0.5045  | 0.3973 | 0.0547  | 0.4271    | 0.0743  | 0.4363 | 0.0897  | 0.2970     | 0.0532  |
| MuSiQue   | 0.3198 | 0.1615  | 0.1821 | 0.1366  | 0.2572    | 0.0657  | 0.2498 | 0.1799  | 0.2720     | 0.1311  |
| MultiNews | 0.2685 | 0.2421  | 0.2682 | 0.2179  | 0.2763    | 0.2202  | 0.2651 | 0.2300  | 0.2126     | 0.2177  |
| SAMSum    | 0.3652 | 0.3582  | 0.3151 | 0.1215  | 0.3599    | 0.1266  | 0.4217 | 0.1867  | 0.3379     | 0.1650  |
| 2WikiMQA  | 0.4018 | 0.3438  | 0.2602 | 0.0936  | 0.3886    | 0.1396  | 0.2821 | 0.1681  | 0.1334     | 0.0765  |

**Table 4: Ablation study of key design choices for composable KV cache extraction.**

| Ablation Methods                    | Qasper | SAMSum | GovReport |
|-------------------------------------|--------|--------|-----------|
| Base Model                          | 0.4417 | 0.3652 | 0.3274    |
| Anchor Tokens                       | 0.2750 | 0.3201 | 0.1280    |
| C <sup>2</sup> KV-Res. (4x) w/o T.  | 0.1572 | 0.3042 | 0.1318    |
| C <sup>2</sup> KV-Residual (4x)     | 0.3185 | 0.3330 | 0.2802    |
| C <sup>2</sup> KV (4x) Fixed-Pos.   | 0.2827 | 0.3303 | 0.2801    |
| C <sup>2</sup> KV (4x) Info-Leakage | 0.2577 | 0.3406 | 0.2543    |
| C <sup>2</sup> KV (4x) Global-Info  | 0.3722 | 0.3810 | 0.2888    |
| C <sup>2</sup> KV (4x)              | 0.3755 | 0.3904 | 0.2967    |

compression method, SnapKV (4× compression on all layers), is applied prior to KV reuse strategies. Across all reuse methods, directly stacking SnapKV with KV reuse results in severe accuracy loss. This pattern is consistent across tasks and models, showing that generic KV compression, while reducing cache size, is incompatible with non-prefix KV reuse without composability-aware adaptation.

These results motivate our design: compressing KV caches naively further distorts already non-composable KV representations, compounding errors during reuse. KV compression and reuse must therefore be jointly designed within a unified framework that produces KV representations both compressible and composable.

#### 4.8 Ablation Study

We conduct ablation studies to validate the key designs of C<sup>2</sup>KV. Table 4 shows that replacing Structured Information Flow with bidirectional Anchor Tokens causes large accuracy degradation, indicating that unrestricted information mixing breaks KV composability. Directly using the untrained Extractor also fails severely, showing that composable KV representations must be learned.

**Position Re-Alignment.** C<sup>2</sup>KV (4x) Fixed-Pos. disables reuse-time positional re-rotation and consistently reduces accuracy, validating the necessity of position re-alignment after KV concatenation.

**Structural Constraints.** Relaxing block-local constraints through Info-Leakage or Global-Info attention harms performance across all tasks. These results show that stable KV reuse requires strict structural isolation and localized information aggregation.

Overall, the results support the core principles of C<sup>2</sup>KV: structured extraction, learned composable representations, and reuse-time positional alignment.

### 5 Related Work

**KV Cache Reuse.** Efficient KV cache reuse is critical for reducing Time-to-First-Token (TTFT) and computational redundancy. *Prefix-based methods*, such as PromptCache [10], PagedAttention [18] and

Radix-Attention [44], cache KV states of common prefixes in a tree structure to avoid recomputation across multiple requests. To handle non-prefix scenarios, *training-free approaches* like CacheBlend [38] and Cache-Craft [1] enable arbitrary text chunk reuse via selective recomputation and KV state alignment, though they may suffer from slight accuracy degradation. *Training-based methods*, including TurboRAG [25] and Block-Attention [26], modify model architectures or employ fine-tuning to support modular cache stitching (e.g., independent attention blocks). While these offer better generation quality, they require significant training overhead and risk compromising the base model’s general capabilities.

**KV Cache Compression.** To mitigate LLM memory bottlenecks, KV cache compression reduces per-token storage. Quantization methods like CommVQ [20] or MiniKV [27] achieve ultra-low bit-precision (e.g., 2-bit) while preserving accuracy. Eviction strategies, such as SnapKV [21] and RocketKV [2], identify and retain crucial tokens via attention-driven scores or reconstruction importance. Furthermore, Merging and Low-rank approaches like Mini-Cache [23] or ReCalKV [32] consolidate redundant KV states across layers or attention heads. These techniques maximize memory efficiency and, when paired with reuse, significantly scale long-context processing.

### 6 Conclusion

In this work, we identify KV cache storage and memory bandwidth as the key bottlenecks in non-prefix KV reuse for long-context LLM inference. We show that existing reuse and compression methods are fundamentally limited by the non-composability of standard KV representations. To address this, we propose C<sup>2</sup>KV, a unified framework that learns a composable and compressed KV cache manifold through a lightweight extractor and structured attention, while keeping the base model frozen. Extensive experiments demonstrate that C<sup>2</sup>KV enables efficient KV reuse and compression without degrading generation quality, substantially reducing inference overhead under long contexts.

### Acknowledgements

This work was sponsored in part by China NSF grant No. 62441236, 62472278, 62432007, 62332014, 62332013, and 62372296, and Shanghai QiYuan Innovation Foundation. This work was partially supported by Alibaba Group through the Alibaba Innovation Research Program and in part by SJTU Kunpeng & Ascend Center of Excellence. The opinions, findings, conclusions, and recommendations in this paper are those of the authors and do not necessarily reflect the views of the funding agencies or the government.

## References

- [1] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation. *Proc. ACM Manag. Data* 3, 3, Article 136 (June 2025), 28 pages. doi:10.1145/3725273
- [2] Payman Behnam, Yaosheng Fu, Ritchie Zhao, Po-An Tsai, Zhiding Yu, and Alexey Tumanov. 2025. RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression. *arXiv preprint arXiv:2502.14051* (2025).
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. [https://proceedings.neurips.cc/paper\\_files/paper/2020/file/1457c0d6bfc64967418bf8ac142f64a-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc64967418bf8ac142f64a-Paper.pdf)
- [4] Junyi Chen, Shihao Bai, Zaijun Wang, Siyu Wu, Chuheng Du, Hailong Yang, Ruihao Gong, Shengzhong Liu, Fan Wu, and Guihai Chen. 2025. Pre\*: Enabling Deterministic Pushdown Automata for Faster Structured LLM Generation. *arXiv preprint arXiv:2506.03887* (2025).
- [5] Junyi Chen, Chuheng Du, Renyuan Liu, Shuochao Yao, Dingtian Yan, Jiang Liao, Shengzhong Liu, Fan Wu, and Guihai Chen. 2025. TokenFlow: Responsive LLM Text Streaming Serving under Request Burst via Preemptive Scheduling. *arXiv preprint arXiv:2510.02758* (2025).
- [6] Ruirui Chen, Weifeng Jiang, Chengwei Qin, Ishaan Singh Rawal, Cheston Tan, Dongkyu Choi, Bo Xiong, and Bo Ai. 2024. LLM-Based Multi-Hop Question Answering with Knowledge Graph Integration in Evolving Environments. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 14438–14451. doi:10.18653/v1/2024.findings-emnlp.844
- [7] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanxuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. 2024. Longrope: Extending llm context window beyond 2 million tokens. *arXiv preprint arXiv:2402.13753* (2024).
- [8] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Barcelona, Spain) (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 6491–6501. doi:10.1145/3637528.3671470
- [9] Chaochen Gao, Xing W, Zijia Lin, Debing Zhang, and Songlin Hu. 2025. LongMagpie: A Self-synthesis Method for Generating Large-scale Long-context Instructions. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=tuA2R6gZEA>
- [10] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems* 6 (2024), 325–338.
- [11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [12] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and Fast Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 10746–10761. doi:10.18653/v1/2025.findings-emnlp.568
- [13] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, Donia Scott, Nuria Bel, and Chengqing Zong (Eds.). International Committee on Computational Linguistics, Barcelona, Spain (Online), 6609–6625. doi:10.18653/v1/2020.coling-main.580
- [14] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37 (2024), 1270–1303.
- [15] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, tiancheng hu, zhang qin, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. 2025. EPIC: Efficient Position-Independent Caching for Serving Large Language Models. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=qjd3ZUIHRT>
- [16] Yiqian Huang, Shiqi Zhang, and Xiaokui Xiao. 2025. KET-RAG: A Cost-Efficient Multi-Granular Indexing Framework for Graph-RAG. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (Toronto ON, Canada) (KDD '25)*. Association for Computing Machinery, New York, NY, USA, 1003–1012. doi:10.1145/3711896.3737012
- [17] Dongkyu Kim, Byoungwook Kim, Donggeon Han, and Matouš Eibich. 2024. Autotag: Automated framework for optimization of retrieval augmented generation pipeline. *arXiv preprint arXiv:2410.20878* (2024).
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 9459–9474. [https://proceedings.neurips.cc/paper\\_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf)
- [20] Junyan Li, Yang Zhang, Muhammad Yusuf Hassan, Talha Chafekar, Tianle Cai, Zhile Ren, Pengsheng Guo, Forozaan Karimzadeh, Chong Wang, and Chuang Gan. 2025. CommVQ: Commutative Vector Quantization for KV Cache Compression. *arXiv preprint arXiv:2506.18879* (2025).
- [21] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM knows what you are looking for before generation. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 722, 24 pages.
- [22] Bin Lin, Chen Zhang, Tao Peng, Hanyu Zhao, Wencong Xiao, Minmin Sun, Anmin Liu, Zhipeng Zhang, Lanbo Li, Xiafei Qiu, et al. 2024. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. *arXiv preprint arXiv:2401.02669* (2024).
- [23] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024. Minicache: Kv cache compression in depth dimension for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 139997–140031.
- [24] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, et al. 2025. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 1160–1183.
- [25] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. 2025. TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 6588–6601. doi:10.18653/v1/2025.emnlp-main.334
- [26] Dongyang Ma, Yan Wang, and Tian Lan. 2025. Block-Attention for Efficient Prefilling. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=7zNY1E2fq>
- [27] Akshat Sharma, Hangliang Ding, Jianping Li, Neel Dani, and Minjia Zhang. 2025. MiniKV: Pushing the Limits of 2-Bit KV Cache via Compression and System Co-Design for Efficient Long Context Inference. In *Findings of the Association for Computational Linguistics: ACL 2025*. 18506–18523.
- [28] Yifan Tan, Haoze Wang, Chao Yan, and Yangdong Deng. 2024. AlignedKV: Reducing Memory Access of KV-Cache with Precision-Aligned Quantization. *arXiv preprint arXiv:2409.16546* (2024).
- [29] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, Anand Rajan Iyer, Tianlong Chen, Huan Liu, Chen-Yu Lee, and Tomas Pfister. 2025. In Prospect and Retrospect: Reflective Memory Management for Long-term Personalized Dialogue Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 8416–8439. doi:10.18653/v1/2025.acl-long.413
- [30] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A Large Language Model for Science. *arXiv:2211.09085 [cs.CL]* <https://arxiv.org/abs/2211.09085>
- [31] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Xunxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2024. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819* (2024).
- [32] Xianglong Yan, Zhiteng Li, Tianao Zhang, Haotong Qin, Linghe Kong, Yulun Zhang, and Xiaokang Yang. 2025. ReCalKV: Low-Rank KV Cache Compression via Head Reordering and Offline Calibration. *arXiv preprint arXiv:2505.24357* (2025).
- [33] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

- [34] An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, et al. 2025. Qwen2.5-1m technical report. *arXiv preprint arXiv:2501.15383* (2025).
- [35] Huan Yang, Renji Zhang, Mingzhe Huang, Weijun Wang, Yin Tang, Yuanchun Li, Yunxin Liu, and Deyu Zhang. 2025. KVShare: An LLM Service System with Efficient and Effective Multi-Tenant KV Cache Reuse. *arXiv preprint arXiv:2503.16525* (2025).
- [36] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. 2025. KVLink: Accelerating Large Language Models via Efficient KV Cache Reuse. In *The Thirtieth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=oDcAGSXZZP>
- [37] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 2369–2380. doi:10.18653/v1/D18-1259
- [38] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kunta Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) (*EuroSys '25*). Association for Computing Machinery, New York, NY, USA, 94–109. doi:10.1145/3689031.3696098
- [39] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 535–546. doi:10.18653/v1/2021.naacl-main.45
- [40] Jiajie Zhang, Yushi Bai, Xin Lv, Wanjuan Gu, Danqing Liu, Minhao Zou, Shulin Cao, Lei Hou, Yuxiao Dong, Ling Feng, and Juanzi Li. 2025. LongCite: Enabling LLMs to Generate Fine-grained Citations in Long-Context QA. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 5098–5122. doi:10.18653/v1/2025.findings-acl.264
- [41] Tianyi Zhang, Jonah Yi, Zhaozhao Xu, and Anshumali Shrivastava. 2024. Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization. *Advances in Neural Information Processing Systems* 37 (2024), 3304–3331.
- [42] Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. 2024. CaM: cache merging for memory-efficient LLMs inference. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (*ICML '24*). JMLR.org, Article 2428, 11 pages.
- [43] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruizi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=RkRrPp7GKO>
- [44] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 62557–62583. doi:10.52202/079017-2000
- [45] Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. ToolQA: a dataset for LLM question answering with external tools. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (*NIPS '23*). Curran Associates Inc., Red Hook, NY, USA, Article 2180, 27 pages.

## A Appendix Outline

In this appendix, we report the following sections as supplementary materials for the main paper.

- Appendix B shows the dataset setup for training and evaluation.
- Appendix C shows the detailed TTFT measurement protocol.
- Appendix D gives full results for accuracy evaluation.
- Appendix E lists the limitations and future work.

## B Dataset Setup

### B.1 Training Setup

We train the C<sup>2</sup>Extractor using supervised fine-tuning on multi-document instruction-following data sampled from HotpotQA, 2WikiMQA, and LongMagpie. We sample 40k instances from each dataset, resulting in 120k total training samples, and train for one epoch only. For LongMagpie, long passages are segmented into multiple documents using sentence- or paragraph-aware splitting.

The base LLM remains frozen throughout training. Only the C<sup>2</sup> token embeddings and per-layer QKV projection heads are trainable. We use AdamW with cosine learning rate scheduling under the following settings: Learning rate: 5e-5, Warmup ratio: 0.06, Weight decay: 0.1, Global batch size: 32, Gradient accumulation: 4 and Precision: BF16.

### B.2 Evaluation Setup

We evaluate C<sup>2</sup>KV on LongBench datasets covering multi-document QA and long-form summarization, including HotpotQA, 2WikiMQA, MuSiQue, MultiNews, SAMSum, QMSum, and GovReport.

For QA tasks, we report F1 following official evaluation protocols. For summarization tasks, we report ROUGE-L. All methods use greedy decoding, with maximum generation lengths of 32 tokens for QA and 512 tokens for summarization.

## C Detailed TTFT Measurement Protocol

This section formalizes the protocol used to measure Time-to-First-Token (TTFT) in our system-level evaluations.

### C.1 Definition and Timing Boundary

TTFT is defined as the elapsed wall-clock time from when a user request arrives to when the language model has its past key-value cache fully prepared on GPU and is ready to start decoding the first output token.

We explicitly exclude the prefill cost of system prompts and query tokens from TTFT for all methods, as their lengths are short and negligible relative to long-context document processing. Offline document KV extraction time is also excluded. All timing measurements are synchronized using `torch.cuda.synchronize()` to ensure accurate GPU timing.

### C.2 Method-Specific TTFT Components

Under this definition, TTFT is measured as follows:

- Full Recompute (FR): TTFT equals the wall-clock time of performing a full prefill over the entire document context to construct the KV cache, since non-prefix document reuse is not supported.
- Load+Blend Methods: TTFT is the sum of loading precomputed document KVs from host memory to GPU and executing GPU-side KV blending or structural transformation kernels. These components are profiled separately and summed.
- C<sup>2</sup>KV: TTFT includes loading compressed document KVs from host memory to GPU and lightweight positional reassignment and RoPE re-rotation. No blending or recomputation is required.

**Table 5: Accuracy comparison on LongBench and GSM8K. Bold and underline denote the best and second-best reuse results, respectively;  $1 \times / 4 \times$  indicate KV compression ratios.**

|                                      | Multi-Document Information Retrieval |               |               |               |               |               |               | Few-shot Learning |               |
|--------------------------------------|--------------------------------------|---------------|---------------|---------------|---------------|---------------|---------------|-------------------|---------------|
|                                      | QA                                   |               |               |               |               | Summary       |               | QA                | Summary       |
|                                      | HotpotQA                             | 2WikiMQA      | MuSiQue       | QMSum         | Qasper        | MultiNews     | GovReport     | GSM8K             | SAMSum        |
| <b>Qwen3-4B-Instruct-2507</b>        |                                      |               |               |               |               |               |               |                   |               |
| FR (1 $\times$ )                     | 0.5392                               | 0.4095        | 0.1991        | 0.2419        | 0.3074        | 0.2386        | 0.2826        | 0.8953            | 0.3846        |
| FR-SnapKV (4 $\times$ )              | 0.3715                               | 0.2930        | 0.0821        | 0.2022        | 0.2055        | 0.2087        | 0.2602        | 0.9007            | 0.3697        |
| Naive (1 $\times$ )                  | 0.2486                               | 0.2035        | 0.0339        | 0.1820        | 0.2521        | 0.2388        | 0.2666        | 0.7448            | 0.3296        |
| Epic (1 $\times$ )                   | 0.3266                               | 0.2412        | 0.0492        | <u>0.2151</u> | <u>0.2697</u> | 0.2364        | 0.2565        | <b>0.8999</b>     | 0.3795        |
| CacheBlend (1 $\times$ )             | <u>0.3487</u>                        | 0.2522        | 0.2049        | 0.1573        | 0.1730        | 0.2398        | 0.1680        | <b>0.8999</b>     | 0.3772        |
| C <sup>2</sup> KV-Res. (4 $\times$ ) | 0.3482                               | <b>0.4419</b> | <u>0.2194</u> | <b>0.2175</b> | 0.2662        | <u>0.2506</u> | <b>0.2830</b> | 0.8837            | <b>0.3883</b> |
| C <sup>2</sup> KV (4 $\times$ )      | <b>0.3801</b>                        | <u>0.3920</u> | <b>0.2300</b> | 0.1824        | <b>0.3344</b> | <b>0.2529</b> | <u>0.2744</u> | 0.8730            | <u>0.3836</u> |
| <b>Llama3.1-8B-Instruct</b>          |                                      |               |               |               |               |               |               |                   |               |
| FR (1 $\times$ )                     | 0.5343                               | 0.4018        | 0.3198        | 0.2452        | 0.4417        | 0.2685        | 0.3274        | 0.6914            | 0.3652        |
| FR-SnapKV (4 $\times$ )              | 0.5045                               | 0.3438        | 0.1615        | 0.2201        | 0.2836        | 0.2421        | 0.2811        | 0.6990            | 0.3582        |
| Naive (1 $\times$ )                  | 0.3973                               | 0.2602        | 0.1821        | 0.2075        | <u>0.3804</u> | <u>0.2682</u> | 0.2573        | 0.4263            | 0.3151        |
| Epic (1 $\times$ )                   | <u>0.4363</u>                        | 0.2821        | 0.2498        | <u>0.2231</u> | <b>0.4109</b> | 0.2630        | 0.2782        | <b>0.7319</b>     | <b>0.4217</b> |
| CacheBlend (1 $\times$ )             | 0.2970                               | 0.1334        | 0.2720        | 0.1967        | 0.2107        | 0.2126        | 0.1860        | 0.6895            | 0.3379        |
| Block-Attn (1 $\times$ )             | 0.4271                               | 0.3886        | 0.2572        | 0.2161        | 0.3433        | <b>0.2763</b> | 0.2595        | 0.6371            | 0.3599        |
| C <sup>2</sup> KV-Res. (4 $\times$ ) | 0.4173                               | <b>0.4526</b> | <u>0.3395</u> | <b>0.2310</b> | 0.3185        | 0.2448        | <u>0.2802</u> | <u>0.7189</u>     | 0.3330        |
| C <sup>2</sup> KV (4 $\times$ )      | <b>0.4828</b>                        | <u>0.4477</u> | <b>0.3587</b> | 0.1885        | 0.3755        | 0.2532        | <b>0.2967</b> | 0.6175            | <u>0.3904</u> |
| <b>Qwen2.5-7B-Instruct</b>           |                                      |               |               |               |               |               |               |                   |               |
| FR (1 $\times$ )                     | 0.5848                               | 0.4332        | 0.3448        | 0.2266        | 0.4358        | 0.2475        | 0.3073        | 0.7991            | 0.4048        |
| FR-SnapKV (4 $\times$ )              | 0.4871                               | 0.3060        | 0.1829        | 0.1988        | 0.2755        | 0.2166        | 0.2858        | 0.8701            | 0.4027        |
| Naive (1 $\times$ )                  | 0.3901                               | 0.2700        | 0.1321        | 0.1820        | 0.3232        | 0.2314        | 0.2716        | 0.5737            | 0.3621        |
| Epic (1 $\times$ )                   | 0.4360                               | 0.2610        | 0.2211        | <u>0.1992</u> | <b>0.3663</b> | 0.2361        | 0.2727        | 0.7212            | <b>0.4113</b> |
| CacheBlend (1 $\times$ )             | 0.1903                               | 0.3244        | 0.2224        | 0.1530        | 0.2001        | 0.2463        | 0.1642        | <b>0.8365</b>     | 0.3063        |
| C <sup>2</sup> KV-Res. (4 $\times$ ) | 0.4606                               | <u>0.4655</u> | <b>0.3655</b> | <b>0.2152</b> | 0.3019        | <b>0.2486</b> | <u>0.2812</u> | <u>0.7502</u>     | <u>0.3805</u> |
| C <sup>2</sup> KV (4 $\times$ )      | <b>0.4795</b>                        | <b>0.4718</b> | <u>0.2856</u> | 0.1879        | <u>0.3491</u> | 0.2453        | <b>0.2987</b> | 0.7141            | 0.3754        |

## D Full Results

### D.1 Accuracy Evaluation

We evaluate non-prefix KV cache reuse across multiple models and tasks under a unified setup. KV reuse is applied only to document tokens, while query and generation tokens are always computed online. Block-Attention results are reported only for Llama-3.1-8B-Instruct due to checkpoint availability.

As shown in Table 5, C<sup>2</sup>KV maintains strong accuracy under 4 $\times$  KV compression, consistently approaching Full KV Recompute (FR) across models and tasks. This demonstrates that non-prefix KV reuse can remain accurate when using composable KV representations.

In contrast, directly concatenating standard KV caches causes severe degradation on multi-document QA tasks, revealing the non-composability of vanilla self-attention KV representations. Training-free reuse methods such as EPIC and CacheBlend partially improve over naive reuse but exhibit unstable performance across models and datasets.

Under the same compression ratio, C<sup>2</sup>KV achieves the best or near-best accuracy among reuse-based approaches on most tasks. Compared with generic KV compression methods such as SnapKV, C<sup>2</sup>KV attains comparable or better accuracy while additionally supporting composable KV reuse.

## E Limitations and Future Work

While C<sup>2</sup>KV effectively enables compressed and composable non-prefix KV reuse, it has several limitations that suggest directions for future work. First, C<sup>2</sup>KV focuses on document-level reuse and assumes that reusable content can be identified and extracted offline. Extending the extractor to support online or incremental KV extraction is an important direction. Second, our current design applies a uniform compression ratio across documents and layers. Future work could explore adaptive or content-aware compression strategies that preserve the composability property while allocating compression budgets more flexibly.